DIGITAL MAINTENANCE ORGANISM

PROJECT RESEARCH DOCUMENT

A Physarum-Inspired Autonomous Maintenance Agent for Personal Computing

Classification: Experimental Research

Status: Pre-Development — Research & Architecture Phase

Date: March 2026

Version: 1.1 (Revised)

This document consolidates all research conducted across multiple investigation phases.
All claims are traceable to verifiable sources listed in the bibliography (52 entries).

Table of Contents

Executive Summary

This document presents the complete research foundation for the Digital Maintenance Organism (DMO) project — an experimental software system that applies principles from artificial life, biological chemotaxis, and autonomic computing to the problem of personal computer maintenance.

The core hypothesis is that personal computers accumulate digital waste in two distinct ways. First, long-running software systems undergo software aging — a formally studied phenomenon involving resource leaks, state degradation, and reliability decline. Second, and separately, personal computing generates operational clutter: temporary files, caches, build artifacts, redundant logs, duplicate files, and abandoned media exports that accumulate because no persistent mechanism exists to process them. The DMO project addresses both problems, but treats them as related yet distinct phenomena that require a bridging architecture, not as a single category.

The proposed solution is a persistent, visually alive maintenance system where Physarum polycephalum-inspired particle agents navigate a two-dimensional projection of the filesystem. The Physarum model governs how the organism moves and how the interface behaves. It does not, by itself, validate the waste classification accuracy or safety guarantees. Those depend on the maintenance intelligence layer underneath — a separate engineering problem addressed in this document on its own terms.

Dual Purpose of This Document

This document serves two purposes, presented together by design. First, it establishes the scientific and technical foundations for an experiment. Second, it articulates the market thesis that justifies conducting the experiment. These are deliberately combined because the experiment's design is shaped by the commercial viability question. The reader should expect both research rigor and product reasoning, explicitly separated where they differ.

Epistemic Transparency

This document classifies major claims into four categories to support rigorous review:

[VERIFIED EVIDENCE] Claims directly supported by cited peer-reviewed literature, published documentation, or verifiable open-source repositories.

[SUPPORTED INFERENCE] Conclusions drawn from verified evidence through logical reasoning. The inference chain is stated; the reader can evaluate it.

[DESIGN DECISION] Architectural or implementation choices made by the project. These are not claims of truth but choices with stated rationale.

[OPEN HYPOTHESIS] Claims that this project proposes but has not yet tested. These are what the experiment exists to validate or refute.

These tags appear inline throughout the document at key claim points. Section 17 provides a consolidated index of all major claims by epistemic category.

1. Problem Statement

1.1 The Dead Ecosystem

[SUPPORTED INFERENCE] Every personal computer is an ecosystem without decomposers. Files are created, modified, cached, duplicated, and abandoned. Applications generate temporary data, build artifacts, log files, and intermediate states. Over months and years, these accumulations consume storage, degrade performance, increase backup times, and create confusion about what is valuable and what is waste.

In biological ecosystems, decomposers — fungi, bacteria, detritivores, scavengers — break down dead organic material and recycle nutrients. Without them, dead matter accumulates and the ecosystem collapses. Personal computers have no decomposers. The operating system provides primitive cleanup mechanisms, but these are manual, periodic, incomplete, and require user initiative.

1.2 Two Distinct Degradation Problems

The DMO project addresses two related but distinct phenomena. Conflating them weakens the scientific foundation; separating them strengthens it.

Problem A: Software Aging (system-state degradation). Software aging is a formally studied phenomenon in software engineering. Research defines it as the progressive degradation of software system performance and reliability over time, manifested as increasing failure rates, growing resource consumption, and declining responsiveness. It is caused by accumulated resource leaks, growing data structures, fragmented storage, configuration drift, and stale internal state. The countermeasure is software rejuvenation — proactive interventions that restore system health.

[VERIFIED EVIDENCE] Software aging is documented in peer-reviewed surveys. The term and its countermeasures are established in the ACM and IEEE literature.

Cotroneo et al. (2014). Software aging and rejuvenation. ACM Computing Surveys.

Pietrantuono & Russo. Software aging in Linux.

Iannillo et al. (2015). Software aging in the Linux kernel. ISSRE 2015.

Problem B: Personal Filesystem Waste Accumulation (operational clutter). Separate from system-state degradation, personal computers accumulate user-facing waste: Adobe temporary files, duplicated downloads, old build folders, abandoned browser caches, stale package manager stores, forgotten media exports, redundant logs. This clutter is not caused by resource leaks or software aging in the formal sense. It is caused by the normal operational lifecycle of applications and user behavior, combined with the absence of any persistent mechanism to process the resulting artifacts.

[SUPPORTED INFERENCE] This problem is widely recognized in the existence of a mature market of cleanup tools (CleanMyMac, Hazel, DaisyDisk, etc.) and in Apple's built-in Storage Management recommendations. The software aging literature does not directly study this category, but the cleanup tool market validates that it is a real and unmet problem for consumers.

Apple. Manage storage on Mac.

The Bridge: The DMO project treats both problems as manifestations of the same underlying absence — no persistent decomposer in the computing ecosystem. Software aging degrades the system internally. Operational clutter degrades the filesystem externally. Both require proactive, adaptive remediation. The DMO is designed to address both, with different strategies for each: rejuvenation-style interventions for system state, and waste classification with safe removal for filesystem clutter.

1.3 What Exists Today and Why It Is Insufficient

[VERIFIED EVIDENCE] Mature maintenance products exist for macOS and other platforms. They are effective at what they do. The strongest incumbents already solve the tasks users most immediately care about: fast scanning, obvious space recovery, duplicate detection, and low cognitive load. The DMO does not claim to be faster at scanning or more efficient at one-time space recovery than these mature incumbents.

Product	Model	Strengths	Limitation for DMO thesis
CleanMyMac X	$39.95/yr	Polished, trusted, comprehensive scanning	Scan-and-clean cycle. No persistence. No adaptation.
Hazel	$42 one-time	Rule-based automation. Beloved by power users.	Requires manual rule creation. No learning.
DaisyDisk	$9.99	Beautiful visual disk map.	Static. No maintenance actions. No organisms.
Gemini 2	$19.95/yr	Strong duplicate detection.	Single function only.
OnyX	Free	Deep system-level maintenance.	Technical users only. No adaptation.
macOS Storage Mgmt	Built-in	Zero-effort, always available.	Static recommendations. No autonomy.

CleanMyMac X

Hazel

DaisyDisk

Gemini 2

[OPEN HYPOTHESIS] The DMO thesis is not "we are faster at scanning." It is that persistent adaptive maintenance with a living visual interface produces better long-term outcomes than periodic scan-and-clean cycles. Specifically: better trust through embodied explainability, better continuous maintenance with less manual intervention, better longitudinal adaptation, and better safety through integrated learning and auditability. This hypothesis is testable but unproven.

1.4 The Core Hypothesis

[OPEN HYPOTHESIS] A personal computer can host a persistent software organism, inspired by the biology and mathematics of Physarum polycephalum, that observes filesystem state,

identifies waste accumulations, navigates toward them using chemotaxis-based algorithms, proposes bounded remediation actions, learns from user feedback, and adapts its behavior over time — all while operating within a strict safety substrate that prevents destructive action. This hypothesis is testable. The experiment this document prepares will either validate it or identify the specific failure points.

2. Theoretical Foundations

2.1 Artificial Life and Digital Evolution

[VERIFIED EVIDENCE] Artificial life (ALife) is the scientific study of life-like behavior in computational systems. Unlike AI, which focuses on cognitive abilities, ALife investigates how self-organization, adaptation, reproduction, and evolution emerge from simple computational rules.

Three foundational properties define digital life:

1. State — the organism possesses internal variables that change over time.
2. Process — rules or algorithms that update the state based on environmental input.
3. Reproduction with variation — the organism can copy itself with mutation, enabling evolution.

[DESIGN DECISION] The initial DMO uses behavioral adaptation (learning from feedback), not biological evolution (mutation + selection + reproduction). True evolution is reserved for the isolated Phase 5 research sandbox. This distinction is critical: the DMO is an adaptive maintenance agent with ALife-inspired embodiment, not a self-replicating digital organism in the strong ALife sense.

2.1.1 Tierra (1991)

[VERIFIED EVIDENCE] Thomas S. Ray created Tierra as a system where self-replicating machine-code programs compete for CPU time and memory inside a virtual computer. Programs mutate during replication. Natural selection operates. Ray observed the spontaneous emergence of digital parasites and hyper-parasites — phenomena that were not programmed but emerged from evolutionary dynamics.

Ray (1991). An Approach to the Synthesis of Life.

Tierra source code (GitHub mirror)

2.1.2 Avida

[VERIFIED EVIDENCE] Avida is an open-source scientific software platform for experiments with self-replicating and evolving computer programs, developed at Michigan State University. Avida organisms evolve logic computations in exchange for faster replication rates. It has been used in hundreds of peer-reviewed studies on digital evolution.

Avida official platform

Avida GitHub

Avida ALife Encyclopedia

2.1.3 Lenia

[VERIFIED EVIDENCE] Lenia is a continuous generalization of Conway's Game of Life by Bert Wang-Chak Chan. By making space, time, and state continuous, Lenia produces self-organizing, self-repairing, moving digital organisms. Over 400 species have been catalogued.

Lenia official website

Lenia GitHub

2.1.4 ASAL: Foundation Models Applied to ALife Search

[VERIFIED EVIDENCE] ASAL (Sakana AI, with MIT, OpenAI, Swiss AI Lab IDSIA, 2024) demonstrates that foundation models can search simulation parameter spaces and discover ALife behaviors across multiple substrates including Boids, Particle Life, Game of Life, Lenia, and Neural Cellular Automata.

ASAL paper (arXiv)

ASAL project page

ASAL GitHub

2.2 Autonomic and Self-Healing Computing

[VERIFIED EVIDENCE] IBM's autonomic computing initiative formalized self-managing systems around four properties: self-configuration, self-healing, self-optimization, and self-protection. Self-healing systems detect, diagnose, and recover from unhealthy states. Research surveys note that exceptional cases may still require human support.

IBM Autonomic Computing Overview

Psaier & Dustdar. Self-healing systems survey.

2.3 Desktop Agents and Computer-Use Frameworks

[VERIFIED EVIDENCE] Local desktop agents with filesystem access exist as open-source infrastructure today. These are task agents, not evolved organisms, but they prove the actuator layer is viable.

Accomplish: Local desktop AI agent with file management, folder-scoped control, action approval, and logging.

Accomplish GitHub

Open Computer Use: Open-source framework for AI agents with browser automation, terminal access, desktop control.

Open Computer Use GitHub

2.4 Additional ALife Resources

ALIEN: Winner of ALIFE 2024 Virtual Creatures Competition. CUDA-powered, millions of particles.

ALIEN GitHub

ALIEN website

Particle Life: Attraction/repulsion rules producing life-like patterns. Core algorithm ~100 lines.

Particle Life GitHub

SALIS: Tierra-inspired artificial life simulator.

SALIS GitHub

Awesome Artificial Life: Curated ALife resource list.

Awesome ALife GitHub

3. Biological Model: Physarum polycephalum

3.1 Why Physarum, Not Cleaner Fish

The initial analogy for the DMO was a cleaner fish. While evocative, Physarum polycephalum is a far superior biological model for three reasons:

1. It navigates toward food sources using chemotaxis, which maps directly to navigating toward waste concentrations.
2. Its behavior has been rigorously modeled as a multi-agent particle system, providing a direct implementation blueprint.
3. It demonstrates measurable learning (habituation) despite having no nervous system, providing a biological basis for the DMO learning loop.

3.2 Biology of Physarum

[VERIFIED EVIDENCE] Physarum polycephalum is a single-celled organism that can grow to cover several square meters. It navigates toward nutrient sources using chemotaxis, leaves chemical trails that influence future paths, forms efficient transport networks between food sources, self-repairs when damaged, and adapts its morphology in real time.

[VERIFIED EVIDENCE] Nakagaki, Yamada, and Tóth (2000) published in Nature that Physarum placed in a maze formed a network that solved the maze optimally. Subsequent work demonstrated that Physarum networks approximate the Tokyo metropolitan rail system.

Nakagaki et al. (2000). Nature, 407, 470.

3.3 Physarum Learns: The Dussutour Experiment

[VERIFIED EVIDENCE] Dussutour and colleagues at CNRS (2016) demonstrated that Physarum exhibits habituation — the simplest form of learning. When repeatedly exposed to aversive but harmless substances, Physarum learned to cross the barrier to reach food. Critically, a trained Physarum could transfer learned behavior to a naive Physarum through cell fusion.

[SUPPORTED INFERENCE] This provides the biological basis for the DMO learning loop: the organism accumulates experience, changes behavior accordingly, and that learned state can be transferred to a new installation or as anonymized aggregate knowledge.

Boisseau, Vogel & Dussutour (2016). Proc. R. Soc. B.

3.4 The Jeff Jones Mathematical Model (2010)

[VERIFIED EVIDENCE] Jones (2010) published in Artificial Life (MIT Press) a multi-agent particle model where each agent: (1) senses chemoattractant in three directions, (2) turns toward the highest concentration, (3) deposits chemoattractant at current position, (4) moves forward at constant speed. The emergent result: complex, organic transport networks from purely local rules.

[DESIGN DECISION] The Jones model is adopted as the motion and visualization paradigm for the DMO. It governs how organisms move through the filesystem terrain and how the interface

behaves visually. It does not govern the waste classification logic, the safety architecture, or the learning algorithm — those are separate engineering layers.

3.4.1 Parameter Mapping to DMO

Jones Parameter	Biology	DMO Equivalent
Chemoattractant	Chemical deposited by agent/food	waste_score mapped to 2D terrain
Food source	Nutrient concentration	Filesystem waste (cache, temp, logs, duplicates)
Sensor Angle (SA)	How wide the agent looks	Exploration breadth (configurable)
Sensor Offset (SO)	How far ahead the agent looks	Scanning depth (configurable)
Rotation Angle (RA)	How far the agent turns	Navigation smoothness (configurable)
Diffusion	Chemical spread	Waste influence radius on nearby zones
Decay	Chemical evaporation	Waste score reduction after cleanup

Jones (2010). Artificial Life, 16(2), 127–153.

Jones, UWE Repository

Jones (2015). arXiv:1511.05869

Jones (2015). arXiv:1511.05862

3.5 Existing Physarum Implementations

[VERIFIED EVIDENCE] Physarum simulations already run in-browser on WebGL/WebGPU handling millions of particles. No new rendering technology is needed.

Implementation	Technology	Capacity	Relevance
tom-strowger/ physarum-rust	Rust + wgpu (WebGPU)	Millions, M2 MacBook Air	Nearest to DMO stack
SuboptimalEng/ slime-sim	TypeScript + WebGPU	High (compute shaders)	WebGPU reference
maximilianklein/ physarum	WebGL2 (zero deps)	1M+ particles	Proves WebGL2 sufficiency
nicoptere/physarum	JavaScript + WebGL	Standard	Accessible reference
fogleman/physarum	Go	High quality renders	Desktop rendering ref
PHYSARUM (Steam)	Compute shader	Real-time	Market proof (450+ units)

physarum-rust

slime-sim-webgpu

maximilianklein physarum (1M+)

nicoptere physarum

fogleman physarum

PHYSARUM Steam

4. The Digital Maintenance Organism: Definition, Goals, and Analogies

4.1 Definition

[DESIGN DECISION] A Digital Maintenance Organism (DMO) is a persistent software entity that observes the filesystem, identifies waste, navigates toward it using chemotaxis-inspired algorithms, proposes bounded remediation, learns from feedback, and adapts over time within a strict safety substrate. The term DMO is proposed by this project and is not an established scientific category.

[DESIGN DECISION] The DMO is an adaptive maintenance agent with ALife-inspired embodiment and control dynamics. The initial system does not exhibit autonomous reproduction and variation. It is not a digital organism in the strong ALife sense. That distinction is intentional and important: overclaiming weakens the work.

4.2 Three Separable Layers

A critical architectural principle: the DMO comprises three distinct layers that must be designed, built, and evaluated independently.

Layer	Responsibility	Primary Concern	Physarum Role
Cleanup Intelligence	Classify waste, score confidence, propose actions	Accuracy, safety, explainability	None — this is an engineering problem
Agent Motion Logic	Navigate 2D terrain, converge on targets, form clusters	Convergence speed, coverage, noise	Primary — Jones (2010) model governs motion
Visual Embodiment	Render organisms, trails, terrain, proposals to user	Clarity, trust, aesthetic identity	Primary — Physarum aesthetics define the look

The organism visualization is the product's identity and differentiator. Its credibility depends entirely on the maintenance intelligence underneath it. A beautiful organism that makes wrong decisions is worse than a boring file list that makes right ones. The visual layer amplifies whatever the intelligence layer produces — which means it amplifies both good decisions and bad ones.

4.3 The Biological Analogy Map

Biological Concept	DMO Implementation
Ecosystem	The filesystem and system state of the personal computer
Decomposer organism	The Physarum-modeled maintenance agent
Food / nutrient source	Waste files: caches, temp files, build artifacts, logs, duplicates
Chemotaxis	waste_score mapped as chemoattractant; agents navigate toward

Biological Concept	DMO Implementation
	high concentrations
Digestion	Processing waste: compression, deduplication, metadata extraction, quarantine, deletion
Pheromone trail	Scan history and chemoattractant deposition on 2D terrain
Habituation (learning)	Accept/reject history modifying future waste_score weights
Knowledge transfer	Exporting learned model to new installation; aggregate community learning
Homeostasis	Maintaining system health metrics within target ranges
Toxin avoidance	Denylist of sensitive paths the organism never approaches
Starvation	Clean system = low chemoattractant = organism becomes quiescent
Symbiosis	DMO benefits user (clean system); user benefits DMO (feedback for learning)

4.4 Goals of the Experiment

This project is an experiment. Each goal is a testable question with a defined failure condition:

1. Classification accuracy: Can waste_score achieve <1% false-positive rate for auto-approved categories? Failure: >5% after learning.
2. Safety: Does quarantine-first prevent data loss in all tested scenarios? Failure: any unrecoverable data loss.
3. Visual coherence: Can users correctly identify problem zones by looking at the organism? Failure: user identification accuracy <70%.
4. Learning effectiveness: Does accuracy measurably improve between week 1 and week 8? Failure: no statistically significant improvement.
5. Performance: Does the daemon + simulation stay below 2% CPU at rest? Failure: sustained >5%.
6. User trust: Do users grant increasing autonomy over time? Failure: trust plateaus or declines after week 2.
7. Ecological viability (Phase 5): Do specialist agents outperform a generalist? Failure: no measurable advantage.

5. Visualization Architecture

5.1 Filesystem to 2D Space: Voronoi Treemaps

[VERIFIED EVIDENCE] Treemaps were invented by Ben Shneiderman for visualizing file system hierarchies. Voronoi treemaps (Balzer & Deussen, 2005) use irregular polygonal cells that produce organic-looking territories proportional to data values.

[DESIGN DECISION] Voronoi treemaps are chosen over rectangular treemaps because organisms navigating rectangular grids look mechanical, not alive. Each folder becomes an irregular organic territory. Waste concentrations appear as chemoattractant hotspots.

Bruls et al. (2000). Squarified Treemaps.

Balzer & Deussen (2005). Voronoi Treemaps. IEEE InfoVis.

d3-voronoi-treemap

d3-voronoi-map

5.2 Visual Element Mapping

[DESIGN DECISION] Every visual element has a direct, non-decorative mapping to real data:

Visual Element	What It Represents
Territory shape	Folder or application zone
Territory size	Actual disk space consumed
Territory color (blue → red)	Health: blue = clean, red = critical waste
Organism position	Current scan focus
Organism trail	Recently scanned path (pheromone trail)
Organism cluster	High-waste zone being evaluated
Organism pulse / glow	Pending action proposal
Organism dimming	Zone already cleaned; low chemoattractant
Bright food node	Waste concentration (cache/temp/log/duplicate)
Territory texture	Fragmentation / disorder level

5.3 Data Pipeline

1. Rust daemon observes filesystem events (notify crate: FSEvents on macOS, inotify on Linux).
2. File metadata classified and scored (waste_score computed per file).
3. Scored data passed to Svelte frontend via Tauri IPC.
4. D3.js Voronoi treemap projects hierarchy into 2D organic territories.
5. Waste scores mapped as chemoattractant concentrations on terrain.

6. WebGL2 Physarum particle system navigates chemoattractant terrain.
7. Cluster detection maps convergence hotspots back to filesystem nodes.
8. User approves/rejects; feedback logged for learning.

notify crate

5.4 Rendering Technology

[DESIGN DECISION] Start with WebGL2 (universal Tauri webview support, proven at 1M+ particles). Upgrade to WebGPU only if performance requires it.

WebGPU fluid simulations reference

6. Waste Classification and Scoring

6.1 What waste_score Actually Is

[DESIGN DECISION] waste_score is a heuristic priority rank between 0.0 and 1.0, not a calibrated probability. It indicates relative likelihood that a file is safe to remove, for the purpose of ordering chemoattractant intensity and proposal priority. It is not a statistical confidence interval. Calibration against a labeled test corpus is a Phase 0 validation task. The score's statistical properties will be formalized during Phase 4 when sufficient user feedback exists to evaluate it.

6.2 The Scoring Formula

waste_score(f) = size_weight(f) × age_weight(f) × type_risk(f) × (1 – recency_score(f))

Component	Formula	Rationale
size_weight	log10(size_bytes / 1024), normalized 0–1	Larger files are higher-value targets. Log scale prevents domination.
age_weight	min(days_since_mtime / 365, 1.0)	Older files more likely stale. Capped at 1.0.
type_risk	Category lookup (see below)	Known waste categories have high empirical cleanup rates.
recency_score	exp(–days_since_access / 30)	Recently accessed = less likely waste. Exponential decay.

Normalization: each component is bounded [0, 1]. The product is also [0, 1]. No additional normalization is applied in v1. If the distribution proves poorly calibrated during Phase 0 testing, sigmoid rescaling will be evaluated.

6.3 Type Risk Categories

[DESIGN DECISION] Initial values are conservative. The learning loop adjusts per user. Values below are starting priors, not final calibrated outputs.

File Category	Risk	Basis
Application cache	0.95	macOS Storage Mgmt classifies as removable [17]
Package manager cache (npm/pip/cargo)	0.95	Designed to regenerate on demand
Browser cache	0.90	All browsers auto-expire and regenerate
Build artifacts (xcode/cargo/webpack)	0.85	Regenerated by rebuild
System logs >30 days	0.80	macOS log rotation default [18]
Temp files (/tmp, app temp dirs)	0.90	POSIX convention: /tmp is ephemeral

File Category	Risk	Basis
Stale downloads (>90d, never opened)	0.60	Moderate — user may have saved intentionally
User documents	0.00	Absolute protection. Never scored.
App bundles	0.00	Absolute protection. Never scored.
System files	0.00	Absolute protection. Never scored.
Databases	0.00	Absolute protection. Never scored.
Cloud sync folders	0.00	Absolute protection. Could cause sync conflicts.

6.4 Diet System: User-Controlled Food Preferences

[DESIGN DECISION] The user configures what the organism eats via a priority queue of food categories with adjustable weights. Disabled categories emit zero chemoattractant — the organisms cannot perceive them. This transforms a settings panel into a feeding interaction.

- "Adobe only" — only Adobe caches are visible to the organism.
- "Friday is Google day" — scheduled diet rotation.
- "Conservative" — only highest-confidence categories active.
- "Aggressive" — all safe categories active, lower proposal threshold.

6.5 Digestion Model

[DESIGN DECISION] Digestion is not just removal. It is transformation. Each food type has a different digestion process:

Waste Type	Digestion Process	Value Extracted
Large log files	Compress (.gz/.zst) before quarantine	Storage reclaimed; content preserved
Duplicates	Keep one, replace others with references	All copies accessible; storage freed
Build artifacts	Record build config, then quarantine	Rebuild instructions preserved
Cache files	Log origin + size, then delete (high conf)	Statistics for learning; storage freed
Temp files	Hash + log, quarantine/delete by age	Audit trail preserved
Stale downloads	Extract metadata, prompt user	Informed user decision

7. Learning and Adaptation Architecture

7.1 Biological Basis

[VERIFIED EVIDENCE] Dussutour (2016) demonstrated Physarum exhibits habituation and can transfer learned behavior through cell fusion. This is the biological basis for the DMO learning loop.

7.2 Signal Quality Hierarchy

[DESIGN DECISION] User feedback is noisy. A rejection may mean bad timing, not "not waste." An approval may be careless. The system treats feedback signals with explicit quality ranking:

Signal	Strength	Weight	Rationale
Restore from quarantine	Strongest negative	−0.40	Delayed evidence of a mistaken proposal. User undid an approved action.
Explicit rejection	Strong negative	−0.20	User examined and refused. May reflect preference or error.
Explicit approval	Moderate positive	+0.05	User agreed. May be informed or careless.
Ignore (no action)	Weak signal	+0.00	Ambiguous. Could mean not seen, not prioritized, or uncertain.
Decision hesitation (>10s)	Uncertainty flag	No weight change	Logged as metadata. Flags category for slower auto-approval.

The asymmetry is deliberate. Trust builds slowly through many approvals and breaks sharply on a single restoration. One restored file from quarantine dramatically reduces confidence for that entire category, not just that specific path.

7.3 Layer 1: Path-Level Memory

Every path accumulates a trust score bounded [−1.0, +1.0]. Scores update per the signal hierarchy above. Paths at −1.0 are permanently avoided. Paths at +1.0 are auto-proposed with minimal delay.

[DESIGN DECISION] Path-level trust is necessary but insufficient. It can overfit on folders that look repetitive but contain context-dependent content (Downloads adjacent to active projects, package caches mixed with custom scripts, Adobe caches near active exports). Therefore trust is conditioned on: path + file type + source application + age profile + size profile. Pure path-based scores are a starting point, not a final architecture.

7.4 Layer 2: Pattern-Level Learning

After hundreds of decisions, the system extracts generalizable signatures: file extension, parent directory category, file age, file size, access recency, application origin, and historical approval rate for similar files. These patterns are the equivalent of Physarum learning which chemical signatures indicate food versus poison.

7.5 Layer 3: Behavioral Adaptation

Learning directly modifies the chemoattractant map. High-trust food emits stronger signal. Rejected zones emit weaker signal or repellent. The visual behavior of the organism reflects its knowledge: a mature DMO moves with purpose toward known-safe zones, avoids rejected areas, and hesitates around unfamiliar territory. The user can see the learning in the movement.

7.6 Knowledge Transfer

Personal transfer: Learned model (trust scores, pattern weights, diet preferences) exportable/importable for machine migration. Analogous to Dussutour's trained Physarum fusing with a naive one.

Community transfer (Phase 5+, privacy-sensitive): Anonymous aggregate approval rates per category. Only statistical patterns, never paths or content. Requires privacy architecture outside initial experiment scope.

7.7 Metadata Exhaust

Every digestion event generates structured data: what was eaten, confidence, user decision, decision time, restoration status, directory context. Over months, this becomes a behavioral profile of digital waste patterns. No company currently possesses such a dataset. This is a research asset.

8. Safety Architecture

8.1 The Safety Pyramid

[DESIGN DECISION] Four mandatory layers. Each must be fully implemented before the layer above activates.

Layer	Function	Status
1: Read-Only Observer	No filesystem modification for minimum 7 days. Audit log begins immediately.	Always active
2: Confidence + Allowlist	No action below 0.80 confidence without user approval. Strict allow/denylist.	Always active
3: Quarantine Gate	Zero permanent deletion. All removals go to quarantine with full metadata. trash crate.	Always active
4: User Interface	All proposals visible. Organism pulses. User approves/rejects. Full undo.	Always active

trash crate (safe reversible deletion)

8.2 The Nearly-Full Disk Problem

[DESIGN DECISION] When disk space is critically low, standard quarantine (copy-then-remove) fails physically. Three solutions, in strict priority order:

Solution 1 (primary): In-place soft-delete. Rename file on same volume to hidden directory + log metadata in SQLite. Zero additional space. The trash crate already works this way on macOS. Restoration is a rename back. This handles the vast majority of cases.

Solution 2 (secondary): External quarantine overflow. If external drive is connected, quarantine overflows to it. Cannot be depended upon.

Solution 3 (exceptional, outside core safety guarantee): Emergency direct deletion. For browser cache and package manager cache only (0.95+ risk score). Direct deletion without quarantine.

Regarding Solution 3: this mode is classified as exceptional. It is opt-in, non-default, and explicitly outside the core safety guarantee. Low-disk conditions are precisely when path assumptions break, applications behave unpredictably, users are stressed, and rollback options are weakest. Emergency mode is acknowledged as a practical necessity that weakens the safety posture. It is never presented as equivalent to normal quarantine operation.

8.3 Filesystem Edge Cases and Dangerous Semantics

[DESIGN DECISION] The following filesystem realities can cause misclassification, failed moves, or data corruption. Each must be addressed during implementation. This section names the hazards explicitly.

Edge Case	Risk	Mitigation Approach
Symlinks	Following a symlink may lead outside the allowed scope	Resolve all links before classification; never follow links to denylisted paths
Hard links	Deleting one reference does not free space if other refs exist	Detect hard link count; warn user; adjust reclaimed-space estimate
macOS aliases	Different from symlinks; Finder-specific	Detect alias resource fork; treat as opaque reference
APFS snapshots	Files may appear deleted but space not freed	Check snapshot retention; warn user about delayed space reclaim
iCloud placeholder files	Look like real files but are cloud stubs	Detect com.apple.icloud extended attribute; never propose for cleanup
.app bundles (macOS)	Appear as folders but are atomic application packages	Detect CFBundleIdentifier; always on permanent denylist
Locked / open files	Move/delete will fail or corrupt active data	Check file locks (flock/fcntl) before any action; skip if locked
Extended attributes	Metadata loss on move between incompatible volumes	Preserve xattrs during quarantine; warn on cross-volume moves
Mounted external volumes	Quarantine may move files to wrong volume	Detect mount points; restrict quarantine to same-volume operations
App sandbox boundaries	Sandboxed apps may own files that look user-accessible	Respect macOS sandbox containers; add to denylist by default
Package-manager stores	npm/pip/cargo stores may be shared across projects	Only propose cache directories, never package source stores
Developer caches (conditional rebuild)	Some caches only rebuild with specific toolchains	Flag as moderate risk; require user confirmation regardless of score

8.4 Action Log Schema

Column	Type	Description
id	INTEGER PK	Unique action identifier
timestamp	DATETIME	When the action occurred
path	TEXT	Original file path
action_type	TEXT	PROPOSED / QUARANTINED / RESTORED / REJECTED / DELETED
category	TEXT	browser_cache / build_artifact / temp_file / etc.
size_bytes	INTEGER	File size before action
confidence	REAL	System confidence (0.0–1.0)

Column	Type	Description
user_decision	TEXT	APPROVED / REJECTED / PENDING / AUTO
quarantine_path	TEXT	Destination path (if quarantined)
sha256	TEXT	File hash for integrity verification
decision_time_ms	INTEGER	User hesitation duration (learning signal)

8.5 Permanent Denylist

The following are invisible to the organism. No waste_score is computed. Zero chemoattractant emitted:

- User documents (~/Documents, ~/Desktop, ~/Pictures, ~/Music, ~/Movies)
- System libraries (/System, /Library, /usr)
- Application bundles (/Applications, ~/Applications)
- Mail stores and email databases
- Database files (.db, .sqlite, .realm)
- Cloud sync active directories (iCloud, Dropbox, Google Drive, OneDrive)
- Virtual machine disks (.vmdk, .vdi, .qcow2)
- Development working trees (.git directories, IDE project files)
- Keychain and credential stores
- Any path the user has explicitly denylisted

9. Privacy and Threat Model

9.1 Data Collected

[DESIGN DECISION] The DMO collects and stores the following data locally. No data is transmitted to any server. All data resides in a local SQLite database.

Data Type	Collected	Stored	Sensitivity	Retention
File paths	Yes	SQLite	Medium — reveals directory structure	Until user purges or uninstalls
File names	Yes	SQLite	Medium — may reveal content nature	Until user purges
File sizes	Yes	SQLite	Low	Indefinite
File types / extensions	Yes	SQLite	Low	Indefinite
Modification / access times	Yes	SQLite	Low	Indefinite
SHA-256 file hashes	Yes	SQLite	Low — irreversible	Until quarantine purge
Application origin	Inferred	SQLite	Low–Medium	Indefinite
User decisions (approve/reject)	Yes	SQLite	Medium — reveals preferences	Indefinite (learning data)
Decision timing (hesitation)	Yes	SQLite	Medium — behavioral signal	Indefinite (learning data)
File contents	NEVER	N/A	N/A	N/A — never read

9.2 Threat Scenarios

Threat 1: Local database compromise. If the machine is compromised (malware, physical access), the DMO database reveals the user's directory structure, application usage patterns, file naming conventions, and behavioral preferences. Mitigation: the database should be encrypted at rest using SQLCipher or equivalent. The encryption key should be tied to the OS keychain (macOS Keychain, Windows Credential Manager).

Threat 2: Quarantine folder as information source. The quarantine directory contains real files moved from their original paths. An attacker with filesystem access gains access to those files. Mitigation: quarantine inherits the file's original permissions. The quarantine folder itself is protected by OS-level user permissions. No additional encryption in v1 (performance cost), but documented as a known risk.

Threat 3: Community learning data leak. If community learning is ever implemented, even aggregate statistical data (approval rates per file type per application) could reveal information about user behavior in aggregate. Mitigation: community learning is out of scope for the initial

experiment. If implemented, it requires k-anonymity or differential privacy guarantees, consent framework, and independent privacy review.

9.3 Privacy Principles

1. No file contents are ever read. The DMO operates on metadata only.
2. No data is ever transmitted to any server. All processing is local.
3. The user can view all stored data at any time through the inspection interface.
4. The user can purge all DMO data (database + quarantine) at any time with a single action.
5. Uninstallation removes all DMO artifacts including the database, quarantine folder, and configuration.

10. Technical Stack

Layer	Technology	Justification
Desktop shell	Tauri 2.0	Rust backend + web frontend. Privacy-first, local, cross-platform.
Backend	Rust	Memory safety, filesystem access, performance. Native to Tauri.
FS observation	notify crate	Cross-platform (FSEvents/inotify/ReadDirectoryChangesW).
Storage	SQLite (rusqlite)	Local, serverless. Metadata cache + action logs + learning data.
Frontend	SvelteKit	Reactive, lightweight, excellent Tauri integration.
2D mapping	D3.js (d3-voronoi-treemap)	Mature Voronoi treemap implementation.
Simulation	WebGL2 (upgrade: WebGPU)	Physarum particle system. WebGL2 for v1 compatibility.
Algorithm	Jones (2010)	Peer-reviewed, proven, open implementations available.
Shaders	GLSL (WebGL2) / WGSL (WebGPU)	Standard for chosen APIs.
Safe deletion	trash crate	Reversible deletion to OS Trash. No direct fs::remove_file.
Rollback	SQLite metadata + in-place rename	Full audit trail with hash verification.
IPC	Tauri commands + events	Native bidirectional Rust ↔ web communication.

Tauri 2.0

notify crate

trash crate

d3-voronoi-treemap

10.1 Simulation-Data Bridge

The most novel engineering: the bridge between filesystem data and the Physarum simulation. This component does not exist anywhere.

1. Filesystem scan → scored nodes (path, size, type, waste_score, category, trust_score).
2. Voronoi treemap projects nodes into 2D. Territory chemoattractant = waste_score × diet_weight × trust_modifier.
3. Physarum agents navigate terrain on GPU. Cluster detection reads density, maps to filesystem nodes.

cluster_position(x,y) → voronoi_cell → filesystem_node → action_proposal

11. Experimental Roadmap

Time estimates assume part-time development by a single developer. All phases sequential except Phase 5 (parallel research track).

11.1 Phase 0: Foundation (2–3 weeks)

Goal: Core infrastructure + waste_score validation.

- Initialize Tauri 2.0 project (Rust + SvelteKit).
- Implement filesystem scanner using notify crate.
- Build metadata extraction: path, size, type, mtime, atime.
- Implement waste_score computation with initial type_risk table.
- Create SQLite schema for snapshots + action logs.
- Develop labeled test corpus: 500+ files manually classified as waste/not-waste across 5 workload profiles.
- Develop formal evaluation protocol: precision, recall, F1 score thresholds per category.

Deliverable: Rust daemon that scans, classifies, scores, and stores results. Command-line only.

Pass criterion: waste_score top-20 candidates match manual classification with >90% agreement. F1 >0.85 for auto-approved categories.

11.2 Phase 1: Static Terrain (3–4 weeks)

Goal: Filesystem rendered as visual terrain. No organisms.

- Connect Rust backend to Svelte frontend via Tauri IPC.
- Implement D3 Voronoi treemap. Map hierarchy to territories.
- Color by waste_score: blue → orange → red.
- Navigation: zoom, pan, drill into subdirectories.

Pass criterion: User correctly identifies red zones matching known waste directories on actual machine.

11.3 Phase 2: Organisms Appear (4–5 weeks)

Goal: Physarum organisms navigate terrain.

- WebGL2 Physarum particle system on Voronoi terrain.
- Chemoattractant from waste_score data.
- Organisms as bioluminescent trails.
- Population scaling: 500–2000.
- No filesystem actions. Pure visualization.

Pass criterion: Organisms converge on same zones terrain colors identify as waste-heavy.

11.4 Phase 3: Proposal System (4–6 weeks)

Goal: Organisms propose, user decides, system executes safely.

- Cluster detection → filesystem node mapping.
- Proposal UI: pulse, tooltip (details, confidence, rationale).
- Approve/reject controls.
- Quarantine execution (trash crate).
- Rollback: one-click restore.
- Digestion model per waste type.
- Diet system configuration.
- Filesystem edge-case handling (Section 8.3).
- Nearly-full disk handling (Section 8.2).

Pass criterion: 100 test cleanup cycles, zero data loss. All quarantined files restorable. Audit log complete.

11.5 Phase 4: Learning Loop (6–8 weeks)

Goal: System adapts from feedback.

- Signal hierarchy implementation (Section 7.2).
- Path + pattern trust scores in SQLite.
- Chemoattractant modification by learned trust.
- Visual behavioral differentiation (confident vs cautious movement).
- Decision-time tracking.
- Inspectable knowledge state UI.
- Personal knowledge export/import.

Pass criterion: Statistically significant accuracy improvement between week 1 and week 8.

11.6 Phase 5: ALife Sandbox (Parallel)

Goal: Evolving agent strategies. Research track only.

- Isolated simulation. Agent parameter genomes.
- Fitness: reclaimed_space ÷ false_positive_rate × approval_rate.
- Selection, mutation, reproduction.
- Strict code boundary from production system.

Pass criterion: At least one evolved strategy outperforms hand-coded type_risk table.

12. Risk Assessment

12.1 Risks That Will Kill the Project

Risk 1: Classification false positives. Mitigation: conservative type_risk values, quarantine-first, 7-day observation, asymmetric trust scoring, labeled test corpus validation in Phase 0.

Risk 2: Visualization overwhelms engineering. Mitigation: safety substrate built first, completely, before visual work.

Risk 3: Two-track confusion. Mitigation: strict code boundary between ALife sandbox and production.

Risk 4: Performance impact. Target: <2% CPU at rest. Mitigation: benchmark early.

12.2 Risks That Will Slow the Project

- WebGPU compatibility (WebGL2 fallback).
- Voronoi treemap performance on large filesystems (hierarchical culling).
- macOS sandboxing restrictions (entitlement configuration).
- Learning cold start (conservative defaults + future community bootstrapping).
- Privacy optics (loud local-only messaging).

12.3 What the Experiment Could Disprove

- Classification accuracy stays >5% false-positive for auto-categories → organism cannot be trusted.
- Users prefer file lists to organism visualization → core differentiator invalid.
- Performance overhead noticeable → system is net negative on health.
- Learning plateaus in days → adaptive behavior claim is false.

Any of these outcomes is valuable. The experiment succeeds by producing actionable evidence.

13. Open Research Questions

1. Cluster detection: How to detect density hotspots in GPU particle system and map to filesystem nodes? Candidate: read density texture, threshold at $\mu + 2\sigma$, spatial join to Voronoi cells.

2. Chemoattractant update frequency: Terrain rebuilds every 60 min or on significant event; chemoattractant values update continuously.

3. Organism population size: 500–2000 scaled to filesystem. Empirically calibrated in Phase 2.

4. Optimal learning rate: Asymmetric rates; validated by convergence measurement in Phase 4.

5. Open-ended evolution: Can maintenance ecology generate useful novelty or converge to fixed heuristics? Fundamental ALife question.

6. Substrate architecture: Daemon vs container vs VM vs strategy engine. Current: Tauri-managed Rust daemon.

7. Multi-agent specialization: Do specialist organisms outperform generalist? Testable in Phase 5.

8. Ground truth methodology: How to construct and maintain a labeled test corpus representing diverse workloads, applications, and user profiles for waste classification evaluation.

14. Glossary

Term	Definition
Artificial Life (ALife)	Study of life-like behavior emerging from computational systems.
Autonomic Computing	Self-managing systems (self-configuration, self-healing, self-optimization, self-protection).
Chemoattractant	Chemical signal organisms navigate toward. In DMO: waste_score mapped to 2D space.
Chemotaxis	Movement in response to chemical stimulus. The Physarum navigation mechanism.
Digital Detritivore	Proposed term: software that consumes digital waste.
DMO	Digital Maintenance Organism. The proposed entity. Not an established scientific category.
Digestion Model	How the DMO processes waste (compress, deduplicate, quarantine, delete).
Diet System	User-configurable food preferences controlling organism perception.
Habituation	Simplest learning: changed response after repeated exposure. Demonstrated in Physarum.
Jones Model	Jeff Jones (2010) multi-agent particle model of Physarum transport networks.
Open-Ended Evolution	Evolution producing continuous novelty rather than converging to fixed solutions.
Quarantine	Recoverable staging area for removed files. Not permanent deletion.
Software Aging	Progressive degradation of long-running software. Formal engineering term.
Software Rejuvenation	Proactive interventions restoring system health before failure.
Voronoi Treemap	Hierarchical visualization using irregular polygonal cells.
waste_score	Heuristic priority rank (0.0–1.0) indicating cleanup safety likelihood.

15. Bibliography and Source Index

52 verified sources. Ordered by first appearance in text. All URLs verified as of March 2026.

[1] Avida official platform.
https://avida.devosoft.org/

[2] Avida GitHub.
https://github.com/devosoft/avida

[3] Avida ALife Encyclopedia.
https://alife.org/encyclopedia/digital-evolution/avida/

[4] Ray, T.S. (1991). An Approach to the Synthesis of Life.
https://tomray.me/pubs/alife2/Ray1991AnApproachToTheSynthesisOfLife.pdf

[5] Ray, T.S. Tierra paper (Georgia Tech).
https://faculty.cc.gatech.edu/~turk/bio_sim/articles/tierra_thomas_ray.pdf

[6] Tierra source code.
https://github.com/bioerrorlog/Tierra

[7] Chan, B.W.-C. Lenia.
https://chakazul.github.io/lenia.html

[8] Lenia GitHub.
https://github.com/Chakazul/Lenia

[9] ASAL (2024). arXiv:2412.17799.
https://arxiv.org/abs/2412.17799

[10] ASAL project page.
https://asal.sakana.ai/

[11] ASAL GitHub.
https://github.com/SakanaAI/asal

[12] IBM Autonomic Computing.
https://public.dhe.ibm.com/s390/zos/vse/pdf3/techconf2004/G07.pdf

[13] Psaier & Dustdar. Self-healing survey.
https://www.infosys.tuwien.ac.at/Staff/sd/papers/Zeitschrift%20Computing%20H.%20Psaier.pdf

[14] Cotroneo et al. (2014). Software aging. ACM.
https://dl.acm.org/doi/10.1145/2539117

[15] Pietrantuono & Russo. Software aging in Linux.
https://wpage.unina.it/roberto.pietrantuono/papers/ESE.pdf

[16] Iannillo et al. (2015). ISSRE 2015.
https://akiannillo.github.io/misc/publications/ISSRE2015_Iannillo.pdf

[17] Apple. Manage storage on Mac.
https://support.apple.com/en-ca/102624

[18] Apple Developer. Logging.
https://developer.apple.com/documentation/os/logging

[19] MacPaw. CleanMyMac X.

https://macpaw.com/cleanmymac

[20] Noodlesoft. Hazel.
https://www.noodlesoft.com/

[21] DaisyDisk.
https://daisydiskapp.com/

[22] MacPaw. Gemini 2.
https://macpaw.com/gemini

[23] Accomplish agent.
https://github.com/accomplish-ai/accomplish

[24] Open Computer Use.
https://github.com/coasty-ai/open-computer-use

[25] Nakagaki et al. (2000). Nature, 407, 470.
https://doi.org/10.1038/35035159

[26] Boisseau, Vogel & Dussutour (2016). Proc. R. Soc. B.
https://doi.org/10.1098/rspb.2016.0446

[27] Jones (2010). Artificial Life, 16(2).
https://pubmed.ncbi.nlm.nih.gov/20067403/

[28] Jones. UWE Repository.
https://uwe-repository.worktribe.com/output/980579/

[29] Jones (2015). arXiv:1511.05869.
https://arxiv.org/abs/1511.05869

[30] Jones (2015). arXiv:1511.05862.
https://arxiv.org/abs/1511.05862

[31] Jeff Jones publications.
https://www.lstmed.ac.uk/about/people/dr-jeff-jones

[32] ALIEN GitHub.
https://github.com/chrxh/alien

[33] ALIEN website.
https://www.alien-project.org/

[34] Particle Life GitHub.
https://github.com/hunar4321/particle-life

[35] Particle Life website.
https://hunar4321.github.io/particle-life/

[36] SALIS GitHub.
https://github.com/PaulTOliver/salis-v2

[37] Awesome Artificial Life.
https://github.com/jetnew/awesome-artificial-life

[38] Bruls et al. (2000). Squarified Treemaps.
https://www.win.tue.nl/~vanwijk/stm.pdf

[39] Balzer & Deussen (2005). Voronoi Treemaps.
https://ieeexplore.ieee.org/document/1532128

[40] d3-voronoi-treemap.
https://github.com/Kcnarf/d3-voronoi-treemap

[41] physarum-rust.
https://github.com/tom-strowger/physarum-rust

[42] slime-sim-webgpu.
https://github.com/SuboptimalEng/slime-sim-webgpu

[43] maximilianklein physarum (1M+).
https://maximilianklein.github.io/showcase/physarum/

[44] nicoptere physarum.
https://github.com/nicoptere/physarum

[45] fogleman physarum.
https://github.com/fogleman/physarum

[46] PHYSARUM Steam.
https://jkhyuen.github.io/physarum

[47] notify crate.
https://github.com/notify-rs/notify

[48] trash crate.
https://github.com/Byron/trash-rs

[49] Tauri 2.0.
https://v2.tauri.app/

[50] WebGPU fluid simulations (Codrops).
https://tympanus.net/codrops/2025/02/26/webgpu-fluid-simulations-high-performance-real-time-rendering/

[51] Particle Life in browser (WebGPU).
https://lisyarus.github.io/blog/posts/particle-life-simulation-in-browser-using-webgpu.html

[52] AgentFS.
https://github.com/tursodatabase/agentfs

16. Revision Log

Version	Date	Changes
1.0	2026-03-11	Initial consolidated research document.
1.1	2026-03-11	Revised per critical review. Major changes: (1) Separated software aging from personal filesystem clutter as distinct problems. (2) Explicitly separated cleanup intelligence, agent motion, and visual embodiment as three independent layers. (3) Added filesystem edge cases section (symlinks, APFS snapshots, iCloud placeholders, etc.). (4) Added privacy and threat model section. (5) Added epistemic classification system. (6) Reclassified emergency direct-delete as exceptional and outside core safety guarantee. (7) Defined waste_score as heuristic rank, not probability. (8) Added signal quality hierarchy for learning feedback. (9) Added path-level trust overfitting warning. (10) Added labeled test corpus and evaluation protocol to Phase 0. (11) Corrected source count (52). (12) Softened traceability claim.

17. Epistemic Classification Index

Consolidated index of major claims by epistemic category.

17.1 Verified Evidence

- Software aging is a formally studied phenomenon [14, 15, 16].
- Physarum polycephalum navigates via chemotaxis and forms efficient networks [25, 27].
- Physarum exhibits habituation and can transfer learned behavior [26].
- The Jones (2010) multi-agent particle model produces life-like transport networks [27, 28].
- Physarum simulations run in-browser on WebGL2/WebGPU at 1M+ particles [41–46].
- Tierra, Avida, and Lenia demonstrate emergent life-like behavior from simple rules [1–8].
- Self-healing systems detect and recover from unhealthy states [12, 13].
- Desktop agents with filesystem access are practical today [23, 24].
- ASAL demonstrates FM-assisted ALife discovery [9–11].

17.2 Supported Inference

- Personal computers behave like ecosystems without decomposers (analogy supported by software aging + cleanup market existence).
- The Physarum learning model provides a biological basis for DMO behavioral adaptation.
- No existing product combines ALife simulation, filesystem visualization, and maintenance tooling.
- Persistent adaptive maintenance may produce better long-term outcomes than periodic scan-and-clean.

17.3 Design Decisions

- Physarum (Jones 2010) adopted as motion and visualization paradigm.
- Voronoi treemaps chosen for organic spatial projection.
- waste_score defined as heuristic rank, not calibrated probability.
- Quarantine-first with no permanent deletion as default safety posture.
- Emergency direct-delete classified as exceptional, opt-in, outside core guarantee.
- Initial DMO uses behavioral adaptation, not biological evolution.
- Three-layer separation: cleanup intelligence / agent motion / visual embodiment.
- Asymmetric trust scoring (+0.05 approve, −0.20 reject, −0.40 restore).

17.4 Open Hypotheses

- The DMO visual interface produces better trust and long-term maintenance than traditional tools.

- waste_score can achieve <1% false-positive rate for auto-approved categories.
- User feedback provides sufficient signal for meaningful behavioral adaptation.
- Physarum convergence patterns can serve as reliable action proposal triggers.
- Multi-agent specialist organisms outperform a single generalist.
- A maintenance ecology can generate open-ended useful novelty.

18. Document Control

Field	Value
Title	Digital Maintenance Organism — Project Research Document
Version	1.1 (Revised)
Date	March 2026
Status	Pre-Development Research Phase
Classification	Experimental Research
Research phases consolidated	Phase 1 (Literature), Phase 2 (Architecture), Phase 3 (Learning Model), Critical Review
Total verified sources	52
Epistemic categories used	Verified Evidence, Supported Inference, Design Decision, Open Hypothesis
Next milestone	Phase 0: Foundation (Rust scanner + waste scorer + labeled test corpus)

End of document.